Semester	B.E. Semester VII – Computer Engineering
Subject	DevOps Lab
Subject In-charge	Dr. Snehal Andhare

Student Name	Vicky Pukale
Roll Number	23102A0033
Grade and Subject Teacher's Signature	

Task Number	2
Project Title:	Secure CI/CD Pipeline for a Loan Application Tracking Portal
Task Title	Agile Planning and DevOps Workflow: Create user stories and acceptance criteria for the Loan Application Tracking Portal. Prepare a product backlog, 15-Task Kanban/Scrum plan, Definition of Done and a DevOps lifecycle diagram from development to operations.
Deliverable:	

1. User Stories and Acceptance Criteria

The Loan Application Tracking Portal MVP revolves around 5 core functional and non-functional epics:

User Story 1: Loan Application Intake & Event Entry (Core Feature 1)

- **User Story:** *As a Loan Officer, I want to submit a new loan application and log state change events, so that the loan journey is recorded in the tracking system.*
- **Acceptance Criteria:**
  - A form must capture: Applicant Name, Email, Loan Amount (in USD/INR), Loan Type (Personal, Home, Auto, Education), and Initial Status (SUBMITTED).
  - The system must auto-generate a unique Application ID formatted as LOAN-XXXX (e.g., LOAN-1001).
  - Form validation must reject empty names or negative/zero loan amounts with descriptive error messages.
  - The newly created application must immediately appear in the database/data store.

User Story 2: Searchable Dashboard & Status Drill-Down (Core Feature 2)

- **User Story:** *As an Applicant or Loan Officer, I want to search applications by ID or Applicant Name and drill down into the timeline, so that I can see the exact progress of the application.*
- **Acceptance Criteria:**
  - The dashboard must provide a search input with real-time or submit-based filtering.
  - Clicking on an application row must reveal a **Status Drill-Down timeline** showing chronological stages: SUBMITTED →|*rightarrow*→ DOCUMENT_VERIFIED →|*rightarrow*→ UNDERWRITING →|*rightarrow*→ APPROVED (or REJECTED).
  - Each stage in the timeline must display its transition timestamp.

User Story 3: Summary Indicators / KPI Metrics (Core Feature 3)

- **User Story:** *As a Branch Manager, I want to view aggregate status summary cards on the dashboard, so that I can instantly evaluate workload distribution.*
- **Acceptance Criteria:**
  - Four distinct indicator cards must be displayed at the top:
    - **Total Applications**
    - **Under Review / In Progress**
    - **Approved Loans**
    - **Rejected / Flagged Loans**
  - Cards must calculate and reflect updated numbers dynamically when a new record or event is added.

User Story 4: Alerts and Exception View (Core Feature 4)

- **User Story:** *As a Credit Analyst, I want an alert view displaying rejected applications and overdue pending requests, so that high-priority exceptions can be reviewed promptly.*
- **Acceptance Criteria:**
  - A dedicated "Alerts / Exceptions" tab or section must filter applications with status `REJECTED` or pending actions.
  - The alert must state the rejection/hold reason (e.g., *"CIBIL Score Below 650"*, *"Incomplete Income Proof"*).
  - The alert row must be highlighted visually (e.g., with a red/amber badge).

User Story 5: Automated & Secure CI/CD Pipeline (DevOps Story)

- **User Story:** *As a DevOps Engineer, I want every code push to trigger automated builds, quality checks, Selenium browser tests, containerization, and deployment, so that defects are caught before production.*
- **Acceptance Criteria:**
  - Git push or PR merge triggers Jenkins via polling or webhooks.
  - Unit and Selenium tests must pass with a **100% success rate** before a Docker image is built.
  - If any test fails, the pipeline aborts immediately, preventing faulty deployments.
  - Successfully built Docker images must be tagged with build numbers and deployed to the target container environment.

2. Product Backlog & 15-Task Scrum / Kanban Plan

The project is structured into **4 sequential Sprints** mapping all 15 tasks into an actionable Agile backlog:

Task #	Sprint	Work Item / Task Title	Type	Story Points	Priority
TASK-01	**Sprint 1**	Problem Definition, Scope & Stakeholder Analysis	Analysis	2	Highest
TASK-02	**Sprint 1**	Agile Planning, User Stories, DoD & DevOps Lifecycle	Planning	3	Highest
TASK-03	**Sprint 1**	Architecture Design, SRS, Data Model & Local Setup	Architecture	5	High
TASK-04	**Sprint 1**	Git Repo Init, Branching Strategy & Skeleton Code	Version Control	3	High

TASK-05	**Sprint 2**	Feature 1 Development (`feature/event-entry`) & PR Flow	Development	5	High	To Do
TASK-06	**Sprint 2**	Complete MVP (Dashboard, Drill-down, Alerts) & Tagging	Development	8	Highest	To Do
TASK-07	**Sprint 2**	Jenkins CI Server Setup & Maven Build Job	CI Automation	5	High	To Do
TASK-08	**Sprint 3**	Declarative `Jenkinsfile` (Pipeline as Code) & Deployment	CI/CD	5	High	To Do
TASK-09	**Sprint 3**	Selenium WebDriver UI Test Suite Development	QA Testing	5	High	To Do
TASK-10	**Sprint 3**	Continuous Testing Quality Gate in Jenkins (Fail & Fix)	CI/CD	5	High	To Do
TASK-11	**Sprint 3**	Dockerization, Multi-Stage `Dockerfile` & Lifecycle	Container	5	High	To Do
TASK-12	**Sprint 4**	Jenkins-Docker Automated CD Pipeline (Image Registry)	CD Automation	8	Highest	To Do
TASK-13	**Sprint 4**	Ansible Playbook for Node Configuration Management	IaC	5	High	To Do
TASK-14	**Sprint 4**	Automated Node Provisioning, Idempotency & Rollback	IaC / Ops	5	High	To Do
TASK-15	**Sprint 4**	End-to-End Verification, Documentation & Viva Defense	Final Release	3	Highest	To Do

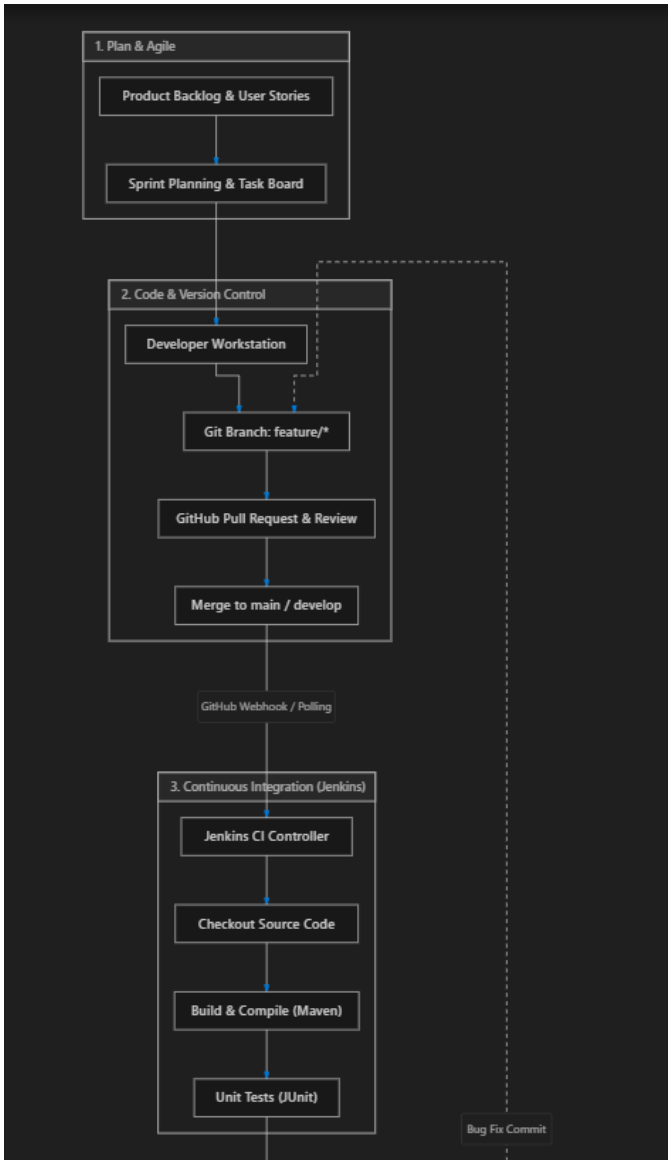
3. Definition of Done (DoD)

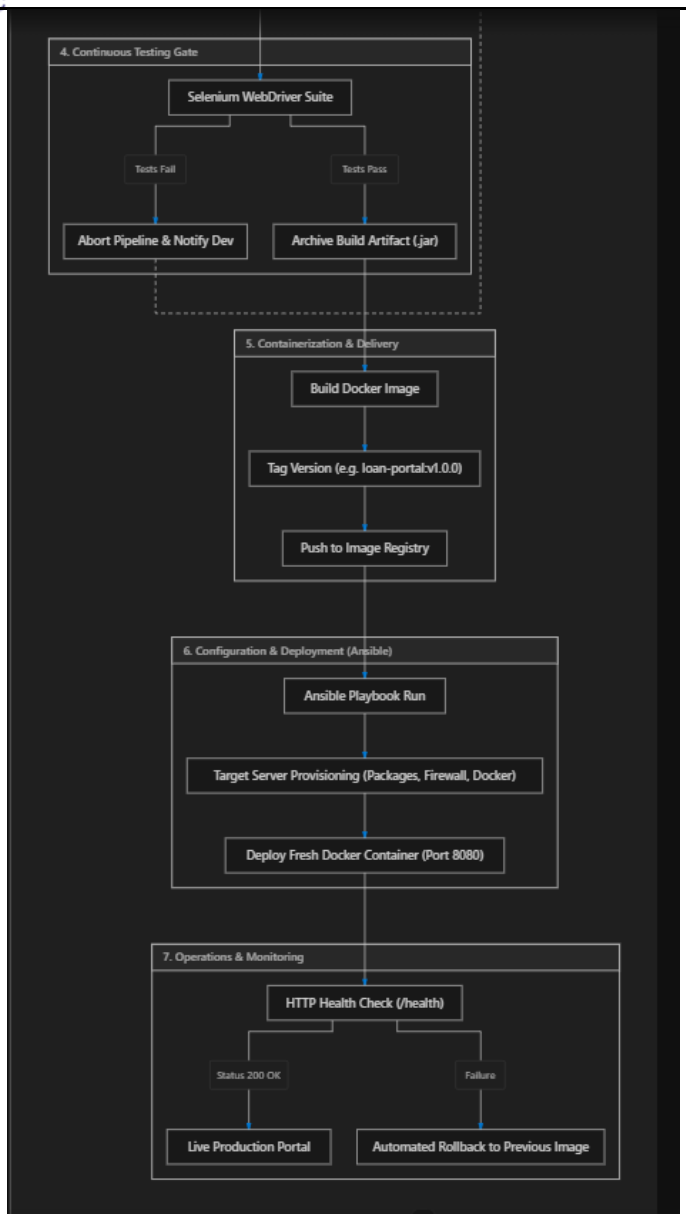
A task or feature branch is considered **Done** only when all the following criteria are fulfilled:

1. **Code Standards:**
   a. Code adheres to clean coding standards (e.g., standard Java / Spring naming conventions or Python PEP 8).
   b. No hardcoded credentials, API keys, or private IP addresses in source code.
2. **Version Control:**
   a. Work is executed on a dedicated feature branch (e.g., `feature/dashboard-view`).
   b. Commits follow semantic commit messages (e.g., `feat:`, `fix:`, `test:`, `ci:`).
   c. Pull Request (PR) merged cleanly into `develop` or `main`.
3. **Automated Testing:**
   a. All unit tests pass locally via Maven (`mvn test`).
   b. Selenium UI regression tests pass in headless mode with zero assertion failures.
4. **CI/CD Integration:**
   a. Jenkins pipeline runs the build stage without errors.
   b. Artifact (`.war` or `.jar`) is compiled and archived.
5. **Containerization & Deployment:**
   a. Application builds into an optimized Docker container image.
   b. Container health check responds with HTTP status `200 OK`.
6. **Documentation:**
   a. README or release notes updated with configuration parameters and port assignments.

4. End-to-End DevOps Lifecycle Diagram

This diagram maps the flow of changes from developer workstation through version control, continuous integration, continuous testing, container packaging, and automated configuration management to production operations:




4. Continuous Testing Gate
Selenium WebDriver Suite
Tests Fail
Tests Pass
Abort Pipeline & Notify Dev
Archive Build Artifact (.jar)
5. Containerization & Delivery
Build Docker Image
Tag Version (e.g. loan-portal:v1.0.0)
Push to Image Registry
6. Configuration & Deployment (Ansible)
Ansible Playbook Run
Target Server Provisioning (Packages, Firewall, Docker)
Deploy Fresh Docker Container (Port 8080)
7. Operations & Monitoring
HTTP Health Check (/health)
Status 200 OK
Failure
Live Production Portal
Automated Rollback to Previous Image